

The Reactor

A structured runtime for local-first applications.

The ask



Three non-negotiables

REQUIREMENT 1

Local-first

Data *and* execution live on the user's machine. Every peer is authoritative. Work continues offline.

RULES OUT

- ✗ A privileged server copy that "wins"
- ✗ Reducers that only run server-side

FORCES

- Command sourcing: ship intent, replay it locally
- One runtime for browser, server, CLI, mobile
- Eventual consistency between peers

REQUIREMENT 2

Append-only

The operation log is a hash-chained audit trail on append-only backends (Swarm, Hypercore). History is never edited.

RULES OUT

- ✗ Reordering or rewriting past operations
- ✗ Deleting operations that turn out stale

FORCES

- Create new operations instead of editing old ones
- Skip values + a garbage-collected view
- Every projection is rebuildable from the log

REQUIREMENT 3

Signed intent

Every Action is signed by its author and must verify on every peer. Nobody's mutation may be altered by someone else.

RULES OUT

- ✗ Transforming other users' operations (OT)
- ✗ Silently discarding a peer's work

FORCES

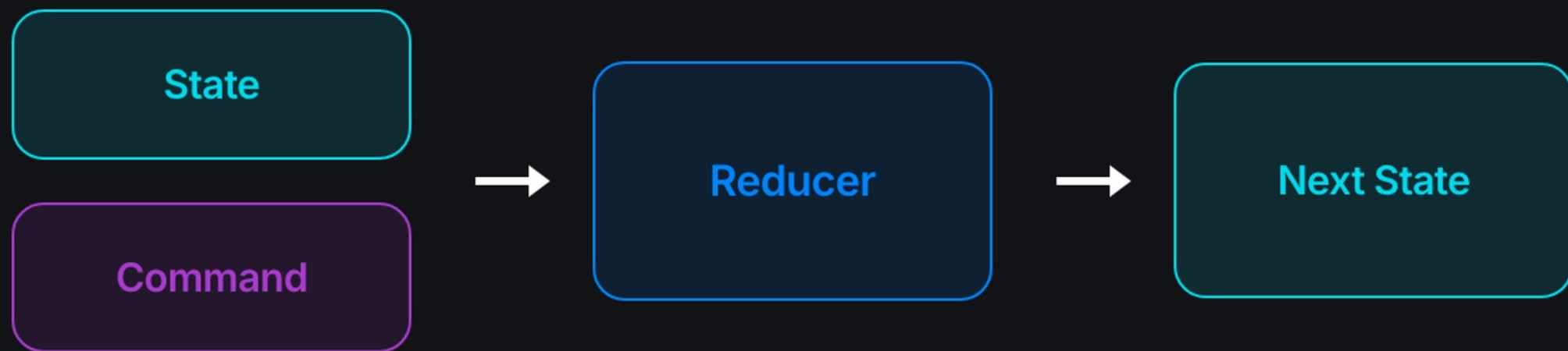
- Replay Actions verbatim; new Operations only
- Signature tiers that can bind state and order
- Dead-letter anything that fails verification

AGENCY AND OWNERSHIP FOR THE USER · THE CONVENIENCE OF CENTRALIZED SAAS · NO MAGIC

Where we started: the reducer

Reducer Update Loop

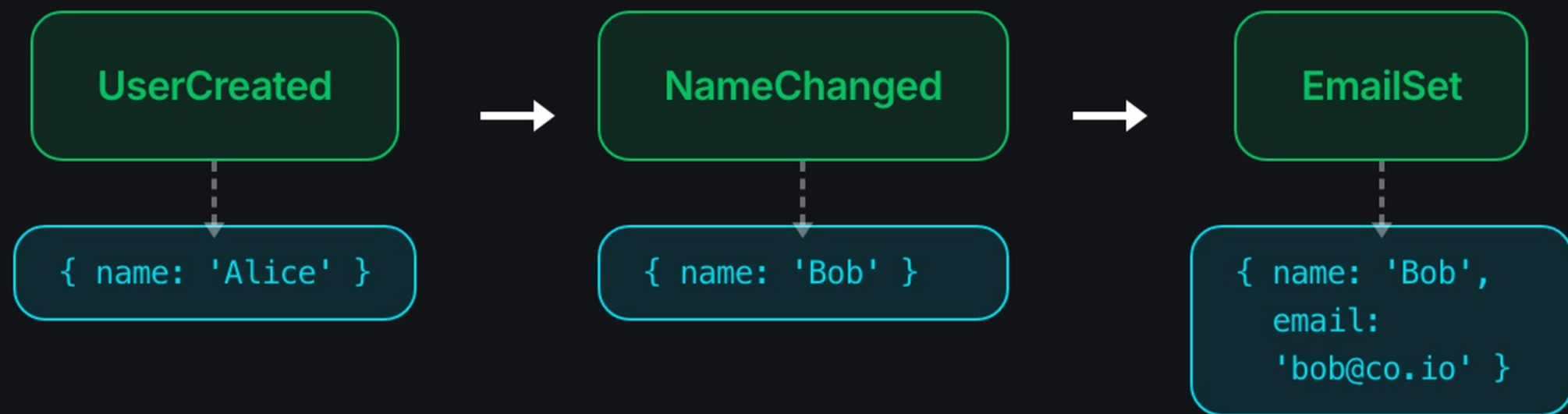
Unidirectional state transitions via pure functions



Event sourcing: state as a stream

Event Sourcing

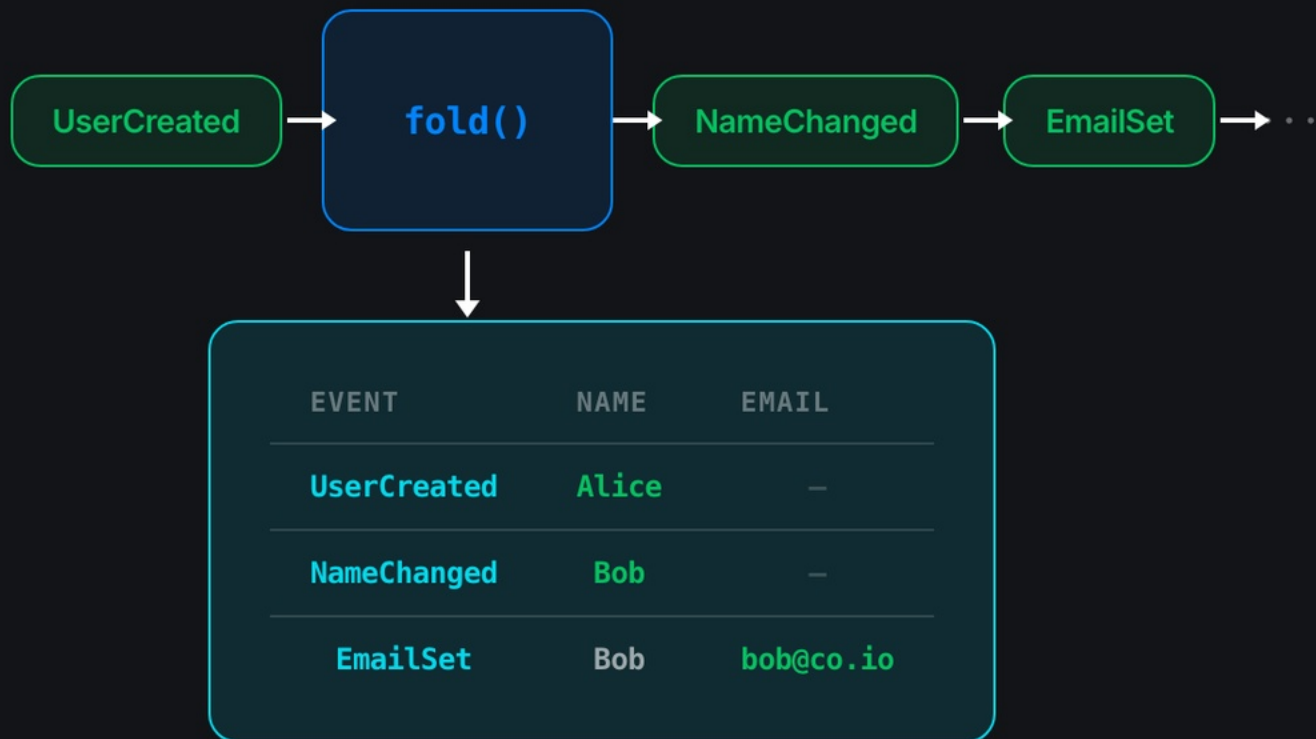
State as a stream of events



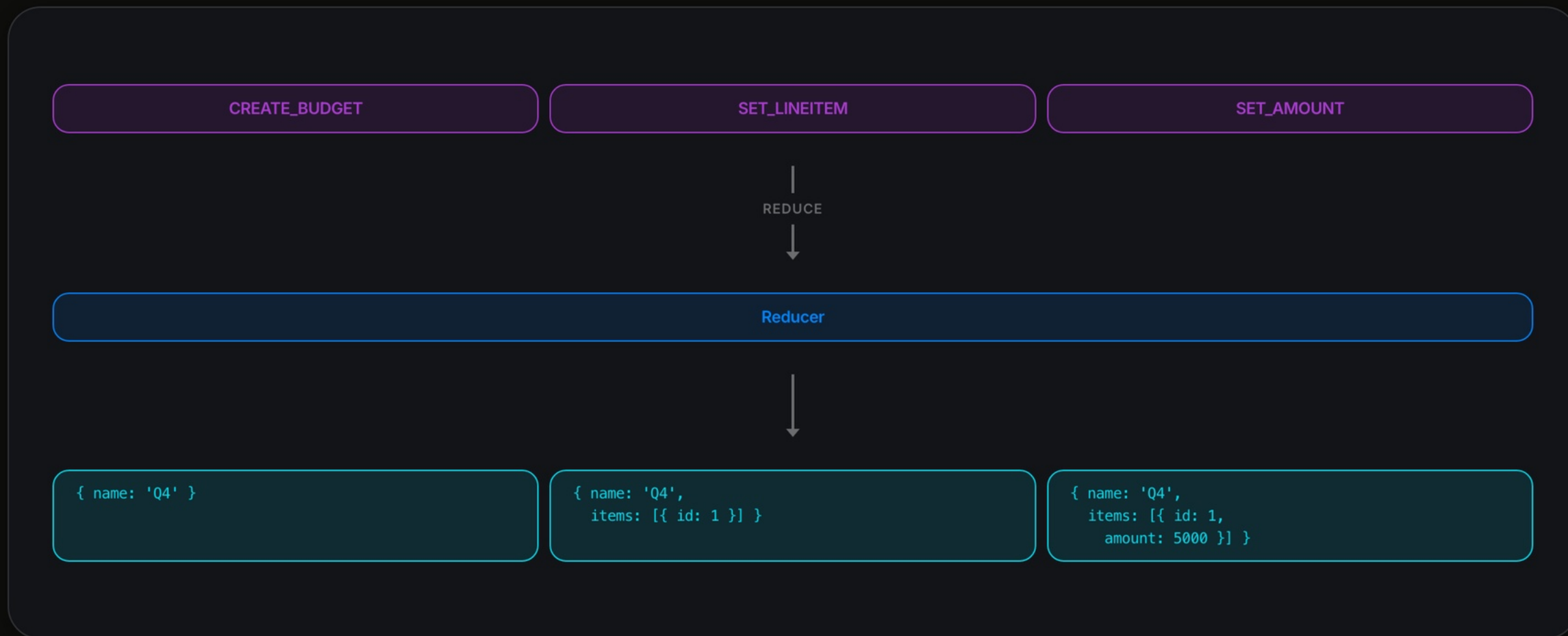
Aggregates: many views from one stream

Aggregate

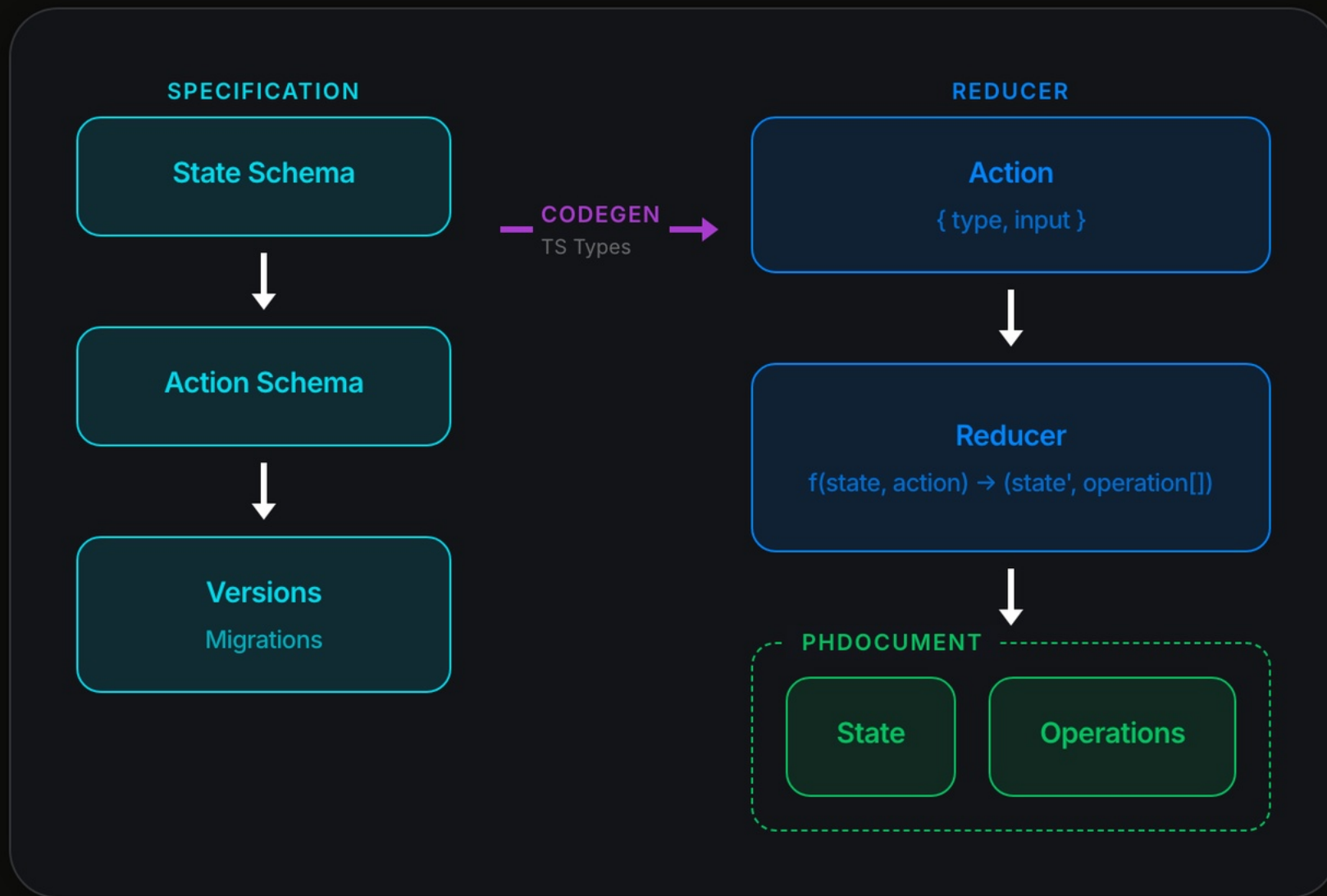
Folding a stream of events into a result



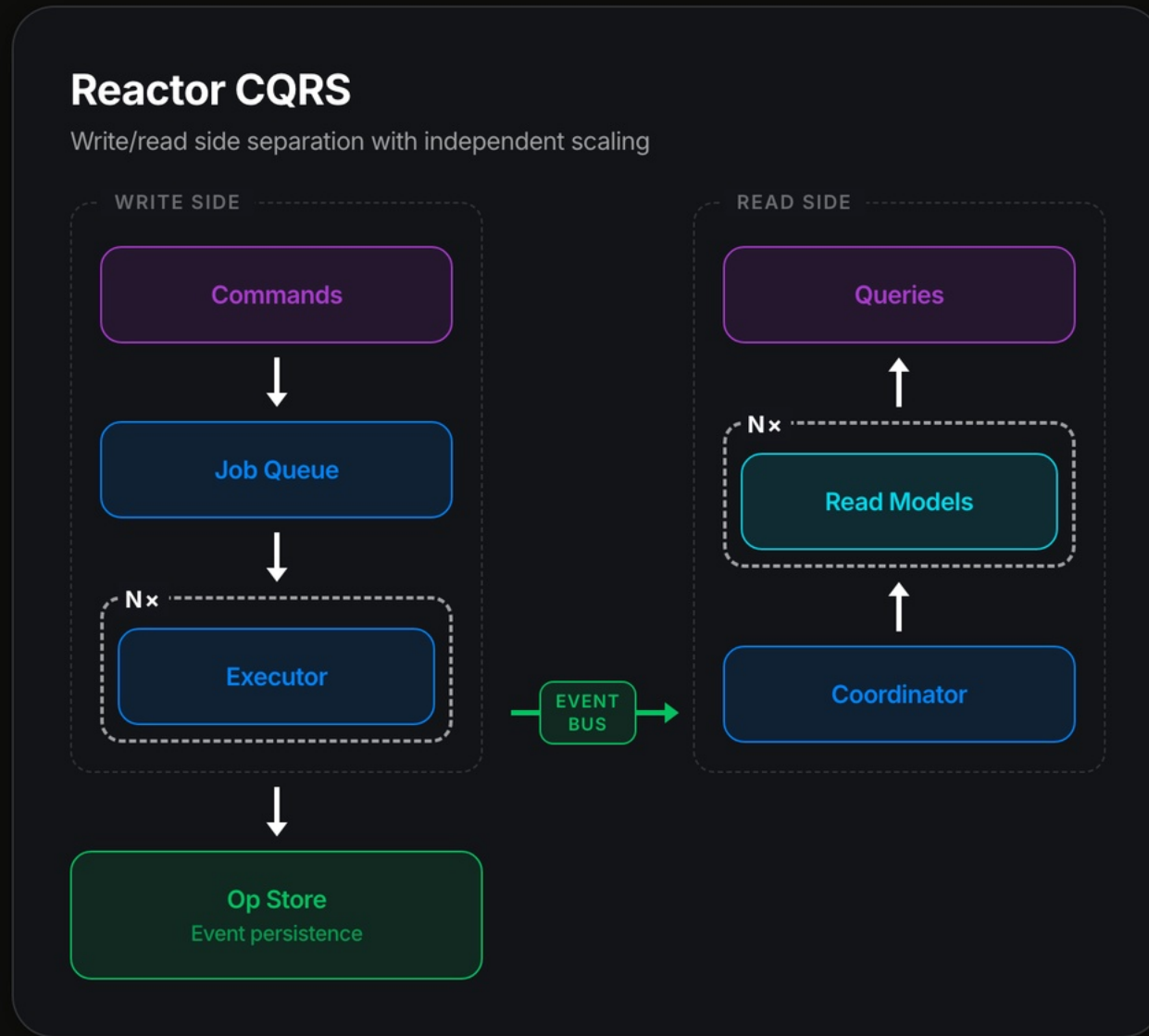
Command sourcing: store the intent, not the result



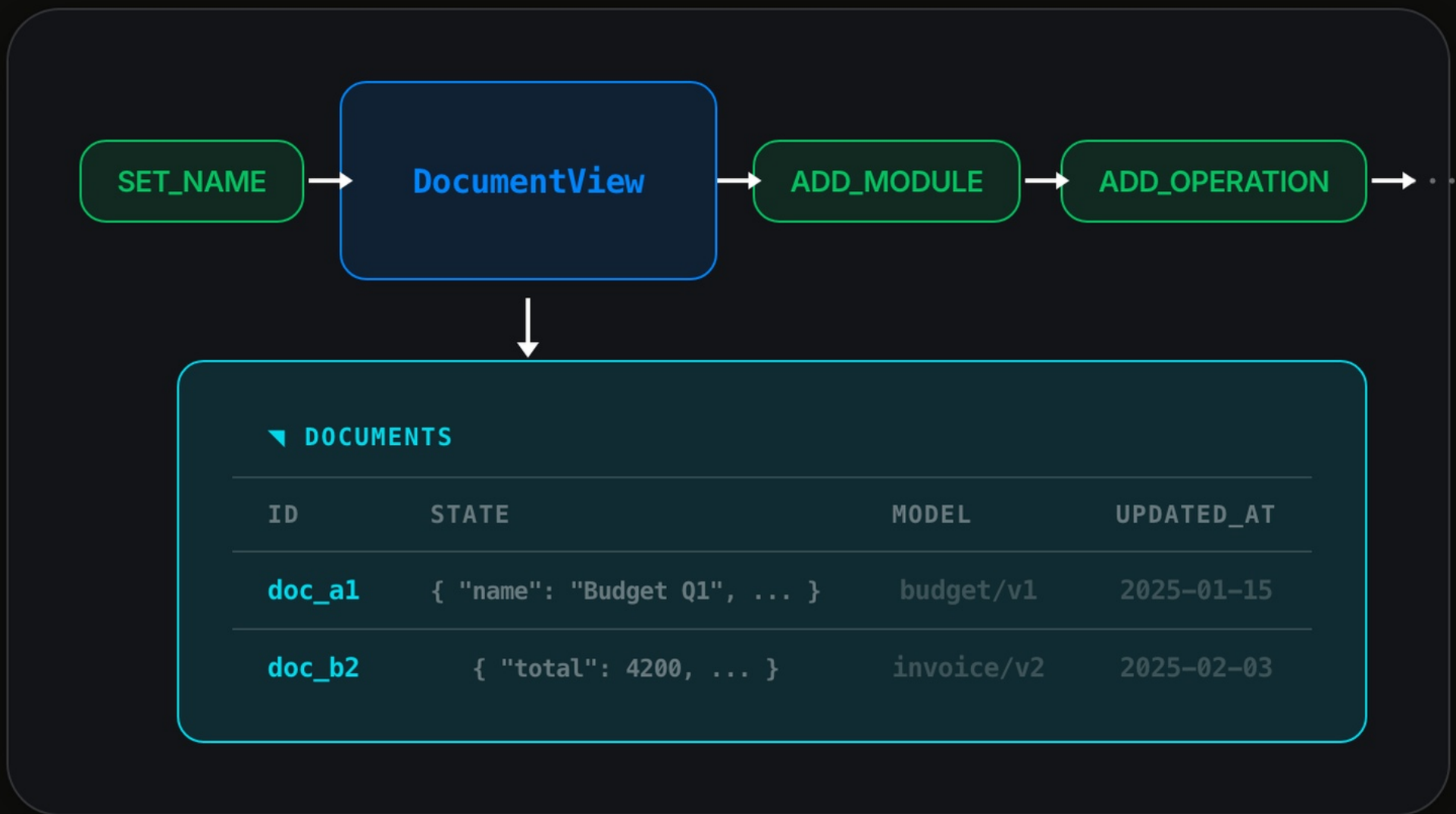
The Document Model



The runtime: CQRS



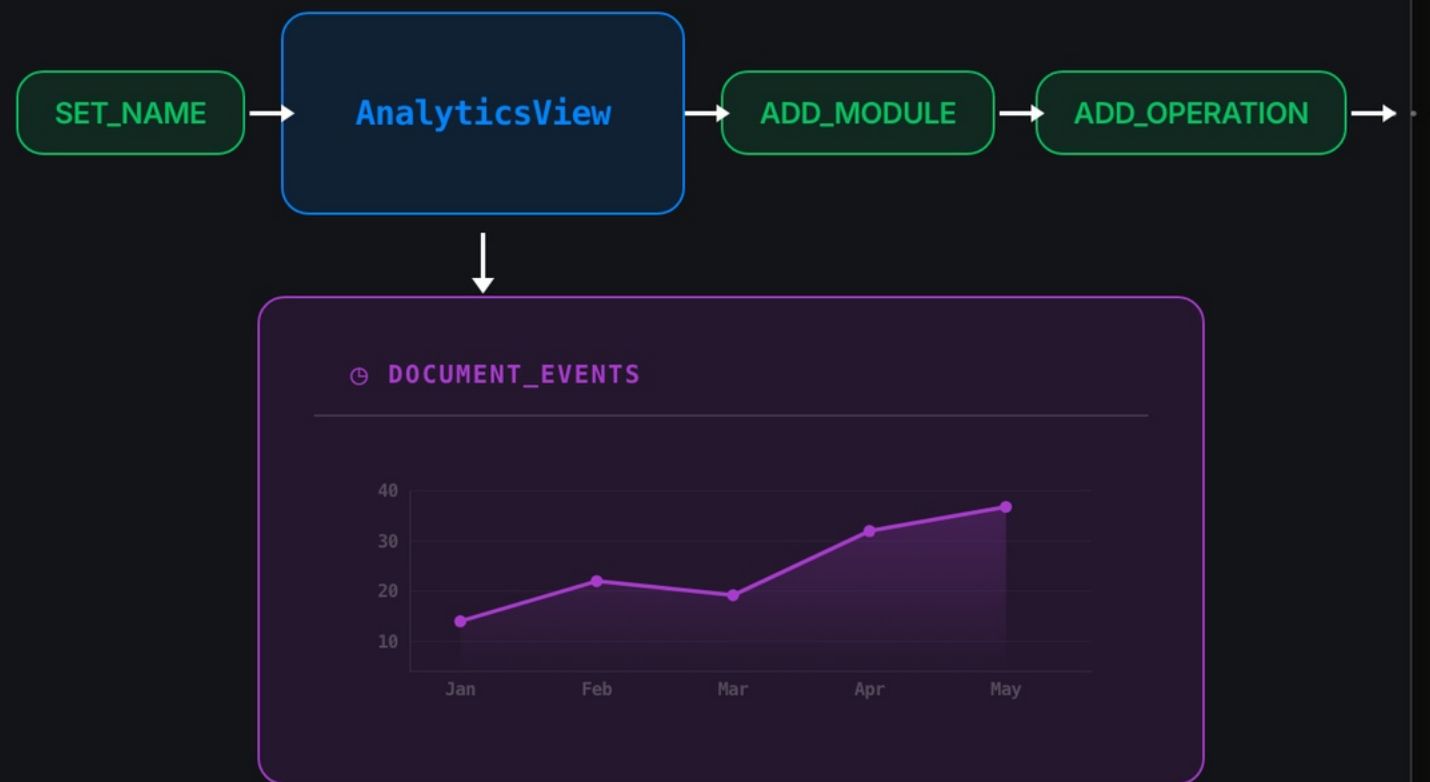
A read model: DocumentView



Read models don't care where they write

Analytics Read Model

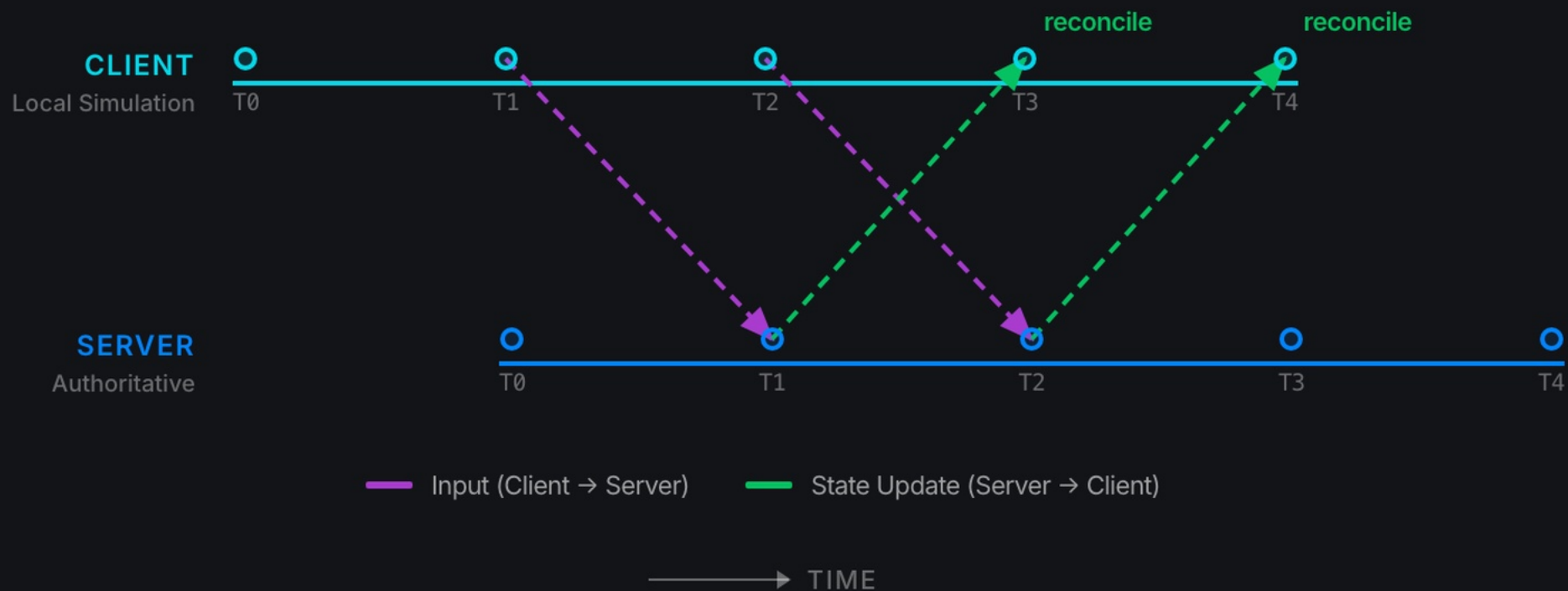
Collecting change analytics into a time-series database



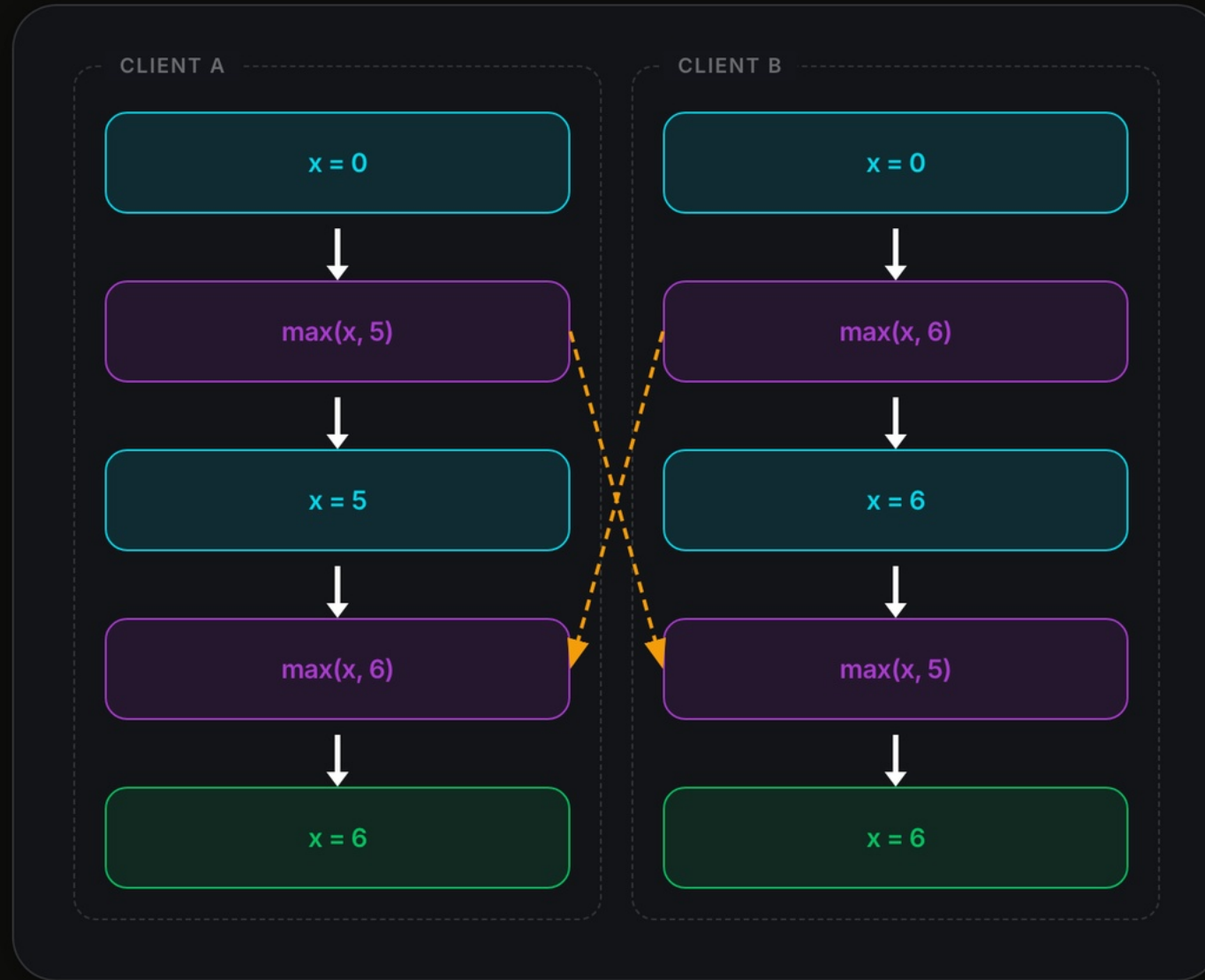
How games do it

Client / Server Game Timelines

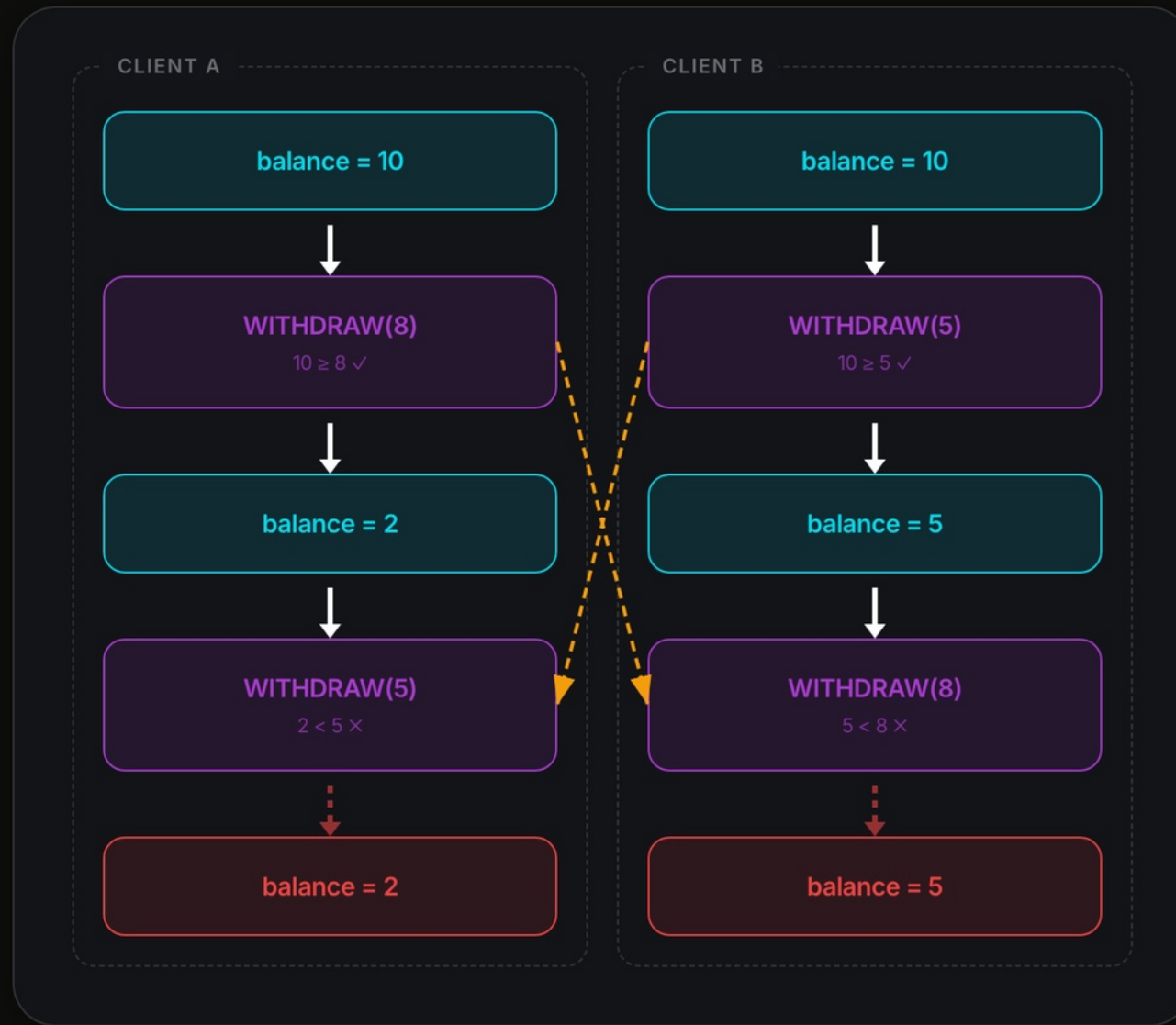
Local simulation runs ahead; authoritative simulation confirms



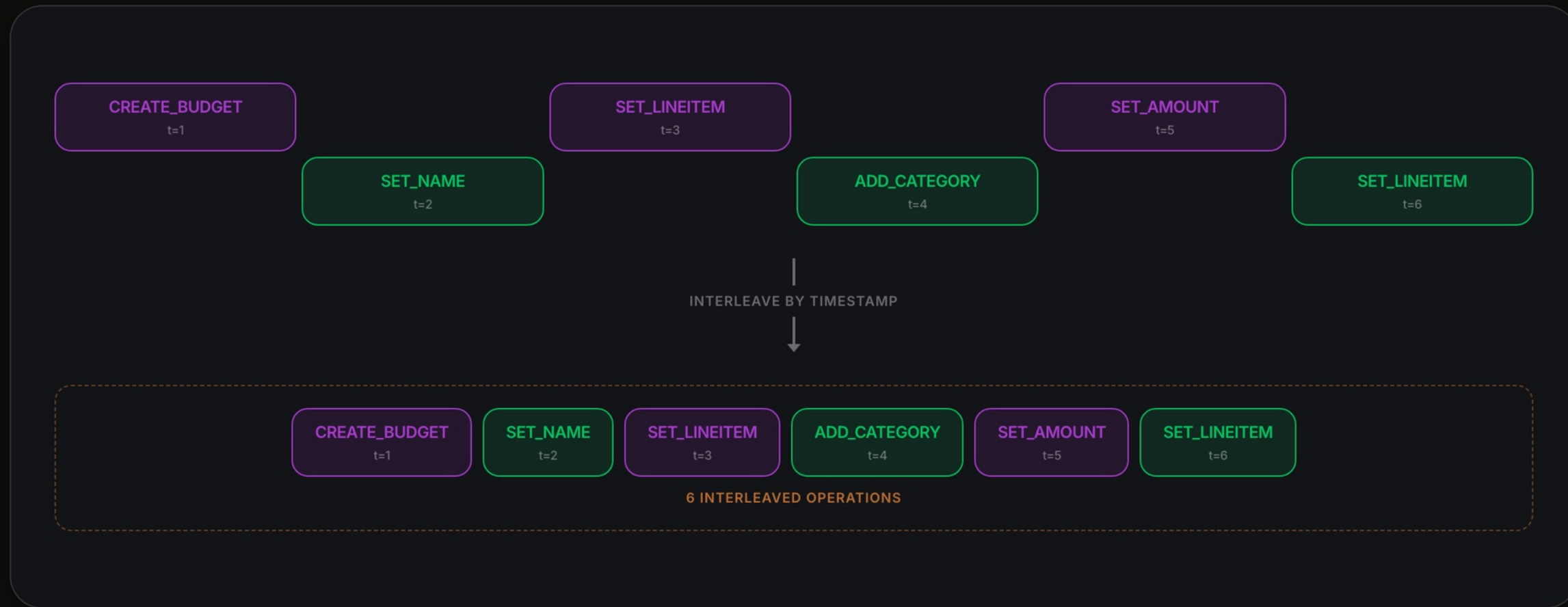
How Figma does it: CRDTs



Where CRDTs stop



Back to the stream: interleave by timestamp



Why you can't just interleave

Interleaving Breaks State Hashes

Each operation records its previous state hash — interleaving invalidates the chain

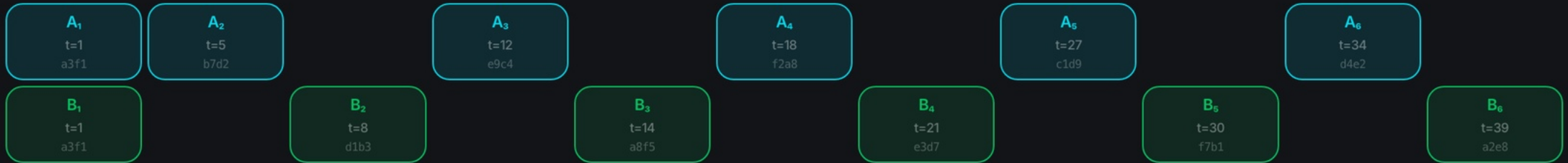


Every operation after the first mismatch must be rewritten with new hashes.

The scorecard

	EVERY PEER AUTHORITATIVE	APPEND-ONLY LOG SURVIVES	INTENT NEVER REWRITTEN	ANY REDUCER (ORDER-DEPENDENT OK)
Authoritative server multiplayer games	 one privileged simulation	 not a concern	 server stomps client work	 any simulation logic
CRDTs Figma-style	 converge without a leader	 ops are applied as-is	 ops are applied as-is	 ops must commute; no balance checks
Operational Transformation Google Docs	 usually server-mediated	 rewrites past operations	 others alter your mutation	 transforms are hard to generalize
Operational Reshuffle The Reactor	 every Reactor reshuffles	 new ops + skip; nothing edited	 Actions replayed verbatim	 replay in order; state-bound sigs if needed

Operational Reshuffle, step 1: sort



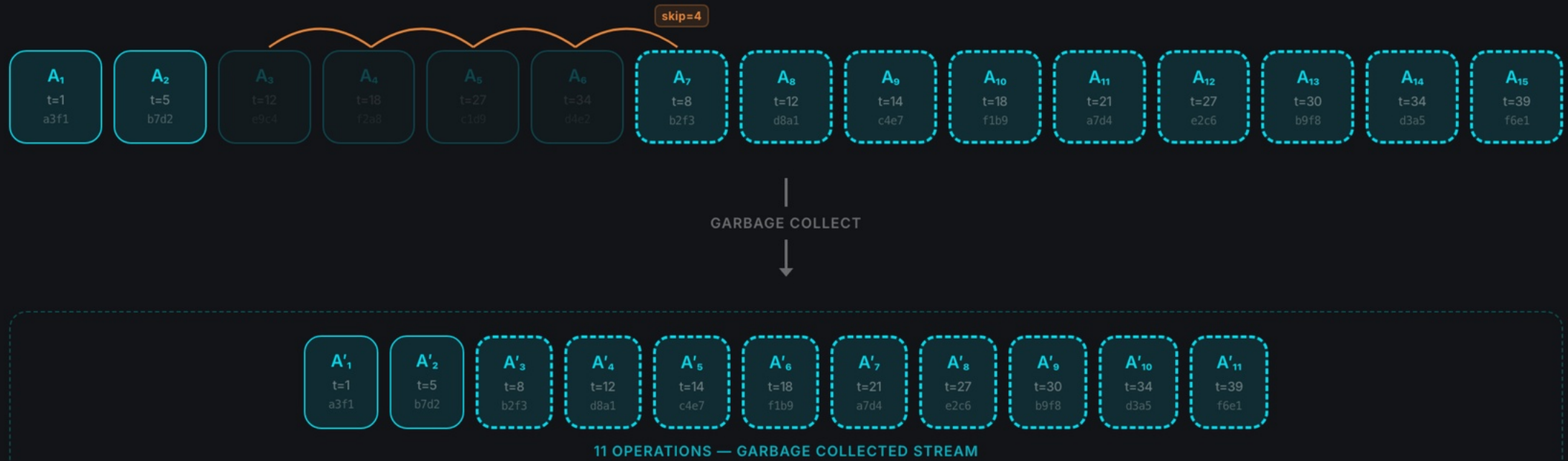
Step 2: find the merge base



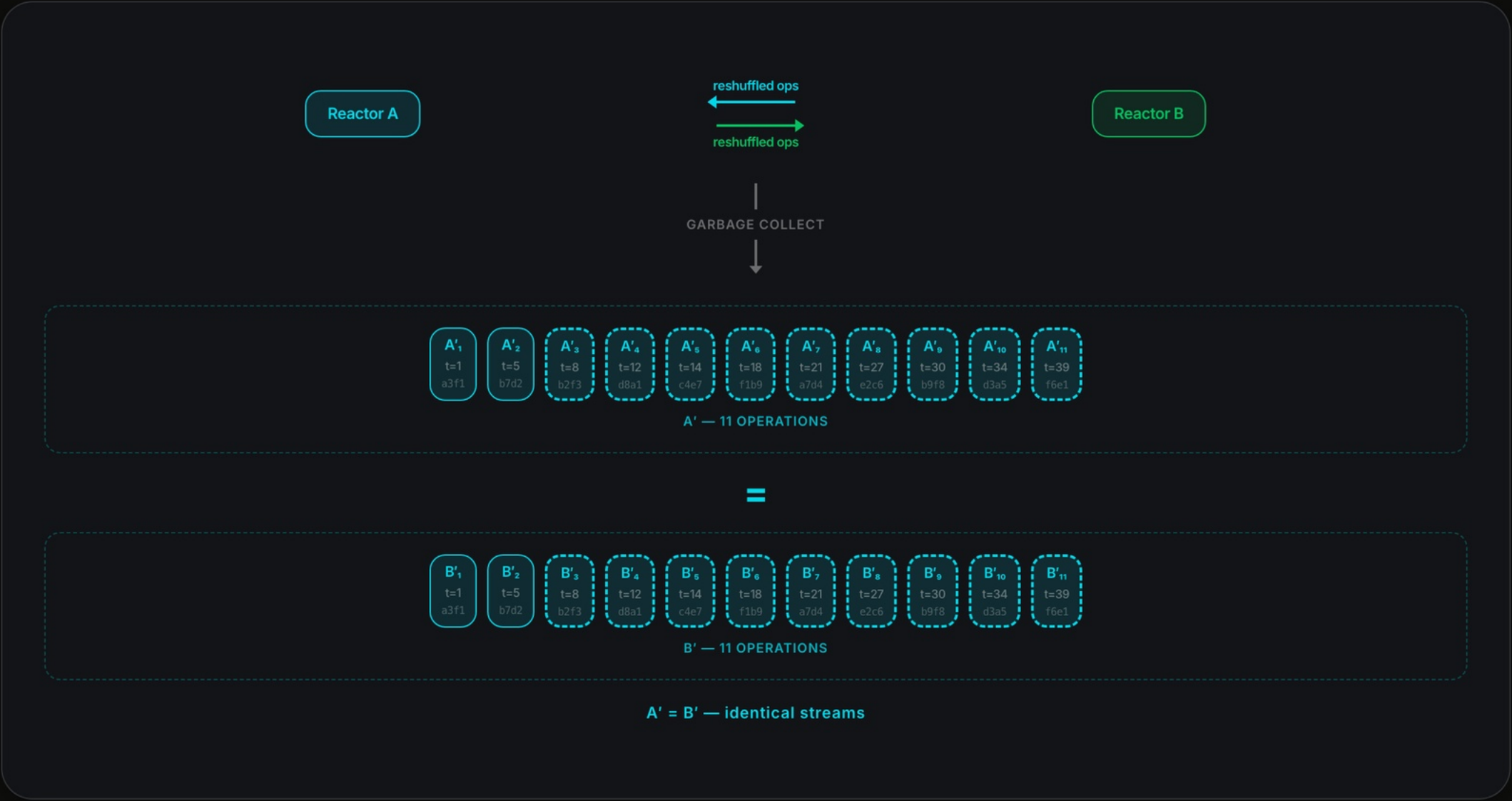
Step 3: replay from the merge base



Step 4: skip, then garbage-collect



Both sides converge



When reordering intent is itself unacceptable

Payload-only signature

DEFAULT

PREVIOUS OPERATION

STATE

+

ACTION

↓

NEXT OPERATION

state

+

```
{
  type: "SIGN_STATEMENT",
  payload: { ... },
  signature: sig(payload)
}
```

✗ Independent of state

signature remains valid even when state changes due to reshuffle

Payload + input-state hash

HIGH SECURITY

PREVIOUS OPERATION

STATE

+

ACTION

↓

NEXT OPERATION

state

+

```
{
  type: "SIGN_STATEMENT",
  payload: { ... },
  signature: sig(payload + H(state))
}
```

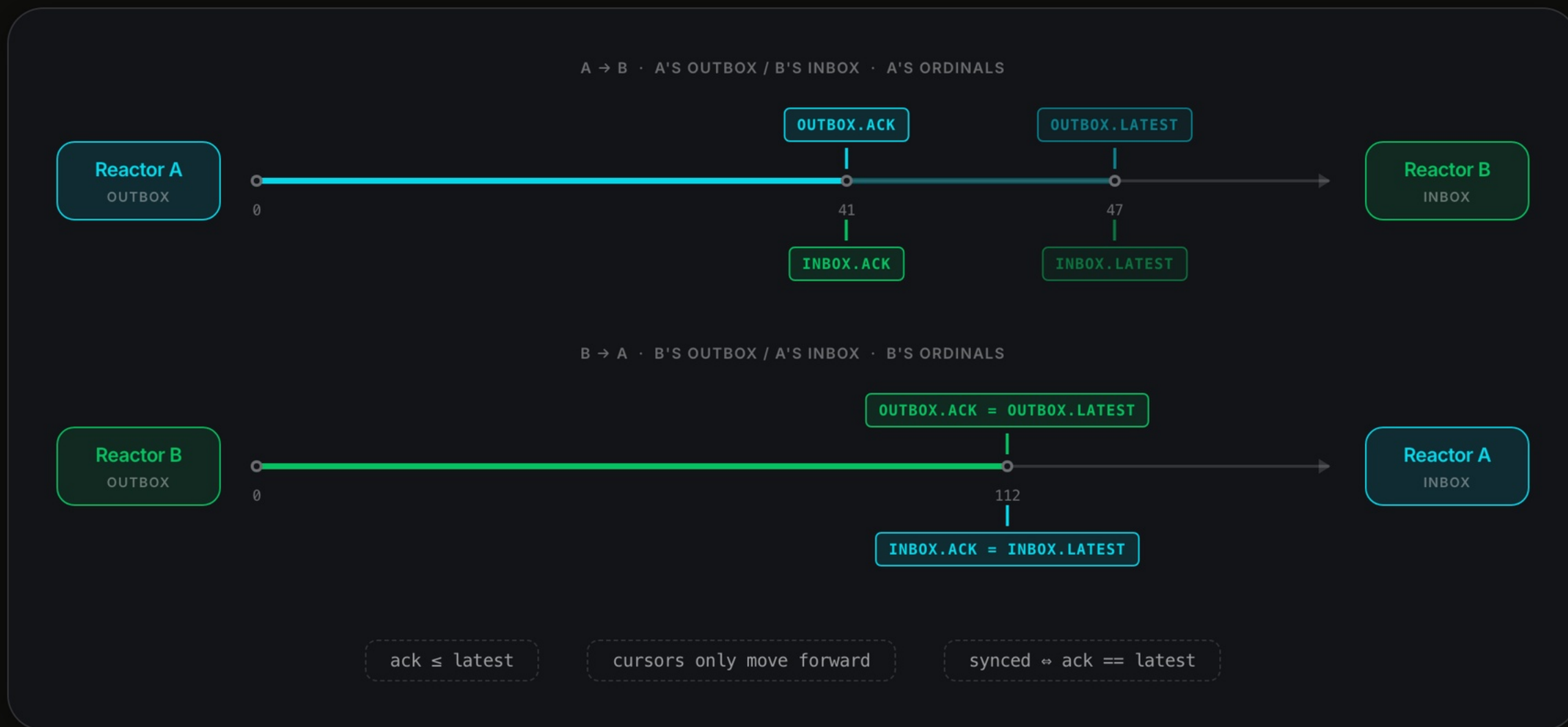
✓ Bound to state

action requires an explicit state

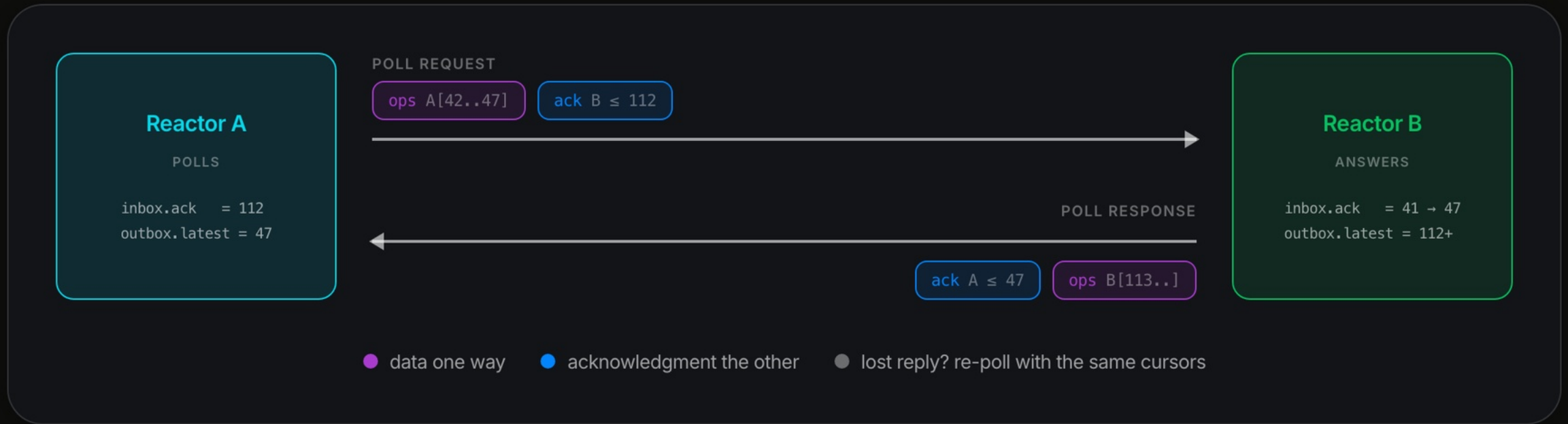
Channels and mailboxes



The entire sync state is four integers



One poll, both directions



The case, restated

	EVERY PEER AUTHORITATIVE	APPEND-ONLY LOG SURVIVES	INTENT NEVER REWRITTEN	ANY REDUCER (ORDER-DEPENDENT OK)
Authoritative server multiplayer games	 one privileged simulation	 not a concern	 server stomps client work	 any simulation logic
CRDTs Figma-style	 converge without a leader	 ops are applied as-is	 ops are applied as-is	 ops must commute; no balance checks
Operational Transformation Google Docs	 usually server-mediated	 rewrites past operations	 others alter your mutation	 transforms are hard to generalize
Operational Reshuffle The Reactor	 every Reactor reshuffles	 new ops + skip; nothing edited	 Actions replayed verbatim	 replay in order; state-bound sigs if needed

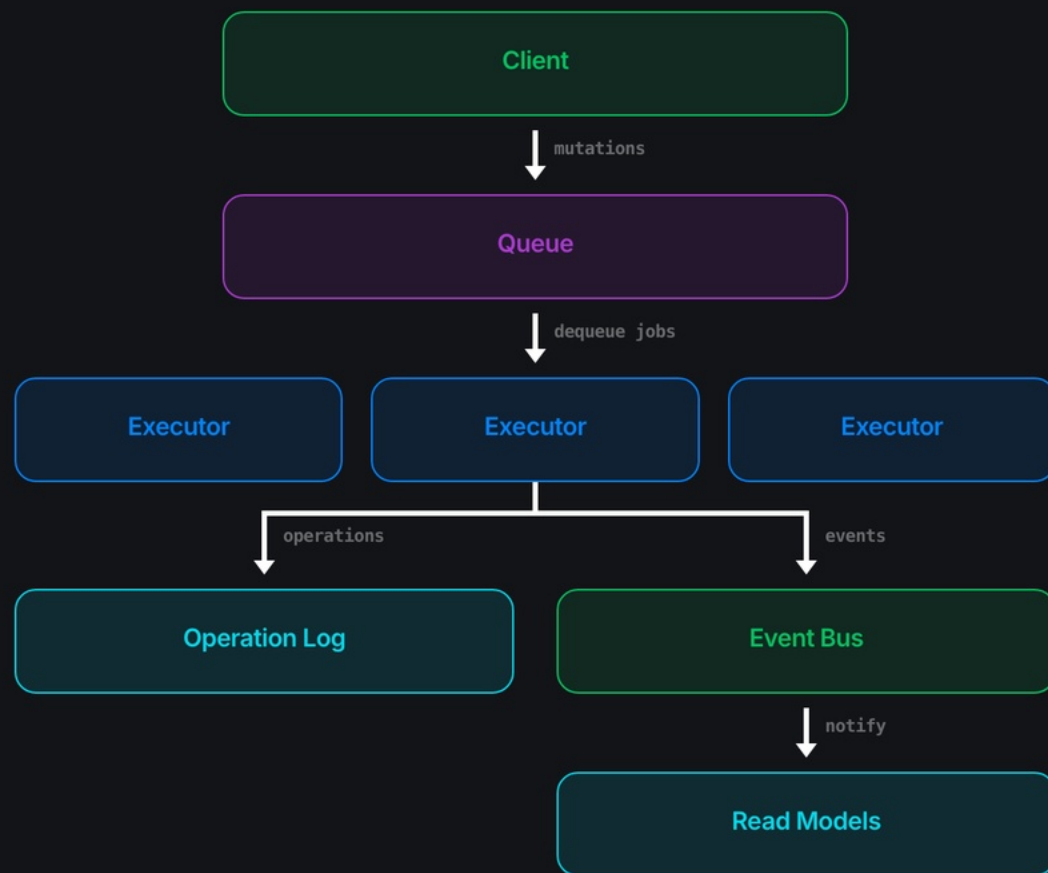
Thanks

Questions?

Reactor job pipeline

Reactor Architecture

Job processing pipeline



Critical-security signature

Payload + input-state hash + id

CRITICAL SECURITY

PREVIOUS OPERATION

id

state

+

action

NEXT OPERATION

id

state

+

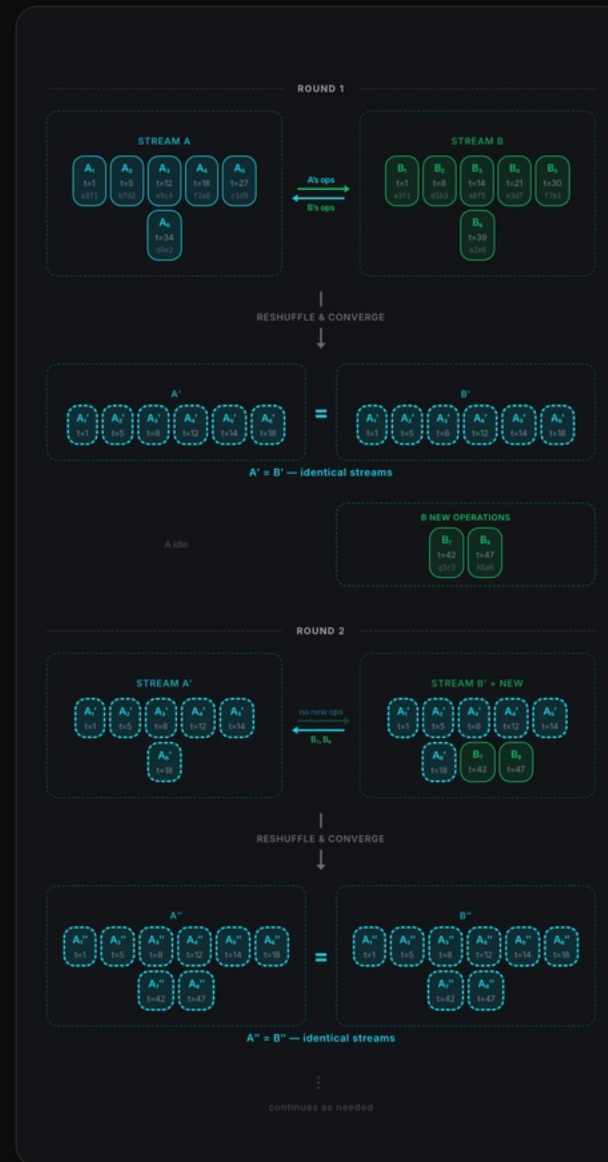
```
{
  type: "SIGN_STATEMENT",
  payload: { ... },
  signature: sig(payload + H(state) + id)
}
```



Bound to state and order

action requires specific state and order

Ping-pong: rounds of convergence



One-way sync

